Department of Energy Engineering, Sharif University of Technology
Term paper of the Intelligent Systems course- Spring 2025

Developing Novel Frameworks for Learning Bayesian Network Structure using Graph Neural Networks

Amirhossein Ranjbarzadeh[1]

[1]M.Sc. Student, Sharif University of Technology, Tehran; amirhranjbarzadeh@gmail.com

Abstract
Probabilistic inference in Bayesian Networks (BNs) is a powerful tool for reasoning under uncertainty, but its application is often limited by the NP-hard complexity of exact algorithms, which become intractable for large, complex networks. This paper investigates the use of Graph Neural Networks (GNNs) as a scalable, data-driven framework to approximate probabilistic inference. A large-scale synthetic Bayesian Network has been constructed to serve as a ground truth and generate a comprehensive dataset for training and evaluation. Four distinct GNN architectures are developed and compared: a baseline Message-Passing Neural Network (MPNN), a Graph Convolutional Network (GCN), an improved Graph Attention Network (GATv2), and a novel Bayesian Graph Neural Network (BGNN). The models are trained to predict node posterior probabilities given a set of evidence. Our results demonstrate that GNNs can successfully learn a generalized inference function. The GCN model achieved the highest predictive accuracy, exhibiting rapid convergence and the lowest KL-Divergence loss on the test set. The proposed BGNN delivered highly competitive accuracy while also offering a principled method for quantifying epistemic uncertainty—a critical feature for probabilistic tasks. This work establishes that GNNs are a viable and powerful alternative to traditional inference methods, with Bayesian GNNs showing significant promise for developing more reliable and uncertainty-aware intelligent systems.

Keywords: Bayesian network, Graph Neural Networks, Bayesian Graph Neural Networks, Graph convolutional Neural networks, Graph Attention Networks

1. Introduction

Probabilistic graphical models are a cornerstone of modern artificial intelligence, providing a robust framework for representing and reasoning about uncertainty in complex systems. Among these, Bayesian Networks (BNs) are particularly powerful, offering a compact representation of the joint probability distribution over a set of variables through a directed acyclic graph (DAG) [1]. They have been successfully applied to a wide range of domains, including medical diagnosis, system reliability analysis, and bioinformatics. More recently, their application has extended to the assessment of infrastructure resilience, where they can model the complex interplay of factors that contribute to a system's ability to withstand and recover from disruptions. Specifically, BNs can be used to enhance resourcefulness—a key component of resilience—by helping to identify critical failures, prioritize actions, and strategically mobilize resources to accelerate recovery in the aftermath of an extreme event [10].

Despite their expressive power, the practical utility of BNs is often constrained by the computational complexity of probabilistic inference. The task of computing posterior probabilities for a set of query variables given some observed evidence is fundamental to their use. While exact inference algorithms exist, their complexity is NP-hard in the general case, rendering them intractable for large, densely connected networks [1]. Approximate inference methods, such as variational inference or Monte Carlo sampling, offer scalable alternatives but come with their own trade-offs in terms of accuracy, convergence guarantees, and computational overhead. This computational bottleneck remains a significant barrier to the deployment of BNs in many real-time or large-scale applications.

In recent years, the deep learning community has seen the rise of Graph Neural Networks (GNNs), a class of models designed to perform learning directly on graph-structured data. Architectures like Message-Passing Neural Networks (MPNNs) [2], Graph Convolutional Networks (GCNs) [3], and Graph Attention Networks (GATs) [8] have achieved state-of-the-art results by learning expressive node representations through iterative message-passing between neighbors. This operational paradigm bears a strong resemblance to belief propagation algorithms used for inference in graphical models, suggesting that GNNs are a natural fit for learning an amortized inference function. By training a GNN on data sampled from a BN, it may be possible to create a model that can perform rapid, parallelizable inference, effectively bypassing the need for traditional iterative algorithms at runtime.

This paper explores the viability of using GNNs to approximate probabilistic inference in Bayesian Networks. We investigate and compare the performance of several prominent GNN architectures, including a baseline MPNN, a GCN, and the more advanced GATv2 [5]. Going beyond simple point estimates of posterior probabilities, we also propose a novel Bayesian Graph Neural Network (BGNN) framework. This approach aims not only to predict the posterior distributions but also to capture the model's own epistemic uncertainty, a critical aspect for creating reliable and trustworthy intelligent systems [9]. By treating the network's weights as probability distributions, the BGNN provides a more

principled, probabilistic approach to the inference task. This research evaluates these models on a complex, synthetically generated Bayesian Network, analyzing their accuracy, efficiency, and ability to generalize.

The remainder of this paper is structured as follows. Section 2 details the methodology for generating the synthetic Bayesian Network, preparing the dataset, and constructing the four GNN architectures. Section 3 presents the quantitative and qualitative results of our experiments, comparing the models' performance on several key metrics. Finally, Section 4 concludes with a discussion of the findings, the limitations of this work, and promising directions for future research.

2. Methodology

This section details the systematic approach used to construct the experimental framework. It begins with the generation of a synthetic Bayesian Network, followed by the formulation of a data structure suitable for Graph Neural Networks (GNNs), and concludes with the procedure for generating the training and evaluation dataset.

2.1. Synthetic Bayesian Network Generation

To serve as the ground truth for this research, a directed acyclic graph representing a Bayesian Network (BN) was computationally synthesized. The objective was to create a complex, non-trivial network structure rather than a model for a specific real-world case.

The network was generated to contain 50 nodes and 330 edges. The generation process involved sequentially adding nodes to the graph. For each new node, an edge to any existing node was formed with a connection probability of 0.3. This process was constrained to ensure the final graph was fully connected, containing no isolated nodes. Further structural constraints were imposed:

- Each node was assigned a discrete number of states, randomly chosen between 2 and 4.
- The maximum number of parents (in-degree) for any given node was limited to 10.

Following the generation of the graph structure, the Conditional Probability Tables (CPTs) for each node were populated with random values. These values were subsequently normalized column-wise to ensure that the probabilities for the states of each node, conditioned on every possible instantiation of its parent states, sum to one, thereby satisfying the axioms of probability.

2.2. Formulation of Node Features and Labels for GNN

To apply GNNs for probabilistic inference, the BN must be translated into a graph-based dataset with appropriate node features and target labels. A key requirement of GNNs is that all nodes in the graph share a consistent feature vector size.

The dimensionality of the node feature vectors, denoted as D_{max}, was set to the maximum number of states present in any node across the network, which in this case is $D_{max} = 4$.

Input Node Features (X_v): The feature vector Xv for each node v is constructed based on its observation status:

- For an unobserved node, the input feature vector is its prior probability distribution. If the node has fewer than D_{max} states, the vector is zero-padded to meet the required dimension.
- For an observed node, the input feature is a one-hot encoded vector. For instance, if a node with three states is observed to be in its second state, its feature vector would be [0,1,0,0]. If necessary. This method allows for the representation of hard evidence while maintaining a consistent feature structure.

Output Target Labels (Y_v): The target for the GNN at each node v is its posterior probability distribution, $P(v|evidense)$. This label vector also has a dimension of D_{max}.

To facilitate the training process, two additional attributes were associated with each node in the graph data structure:

- **number_of_states**: An integer indicating the true number of states for that node. This allows the model to correctly interpret the output and ignore predictions for padded, non-existent states.
- **observed**: A binary flag indicating whether a node was part of the evidence set for a given sample. This is used to mask the loss function during training, as the GNN is not required to predict the probability distribution for nodes whose states are already known.

2.3. Dataset Generation via Probabilistic Inference

A dataset of 5,000 unique samples was generated for training and evaluating the GNN models. Each sample consists of a graph with a specific set of observed evidence nodes and the corresponding ground-truth posterior probabilities for all nodes.

The sampling strategy for selecting evidence was uniform random sampling, where each node in the network had an equal probability of being chosen as an evidence node in any given sample.

In this study, all probabilistic modeling and inference were performed using pySmile, a Python library that serves as a wrapper for the SMILE (Structural Modeling, Inference, and Learning Engine). SMILE is a high-performance C++ engine developed by the Decision Systems Laboratory at the University of Pittsburgh and is now maintained and developed by BayesFusion, LLC.[1]

The Figure 1, shows the process for generating each sample.

This procedure resulted in a comprehensive dataset of 5,000 unique graph instances, each prepared with the necessary input features, node attributes, and ground-truth labels required for supervised training of the GNN framework.

Figure 2 visualize the synthetic Bayesian Network forming the ground truth for this research. To illustrate the data generation process, this figure highlights nodes 12, 13, and 48 as an example of an evidence set that could be used to create a single training sample.

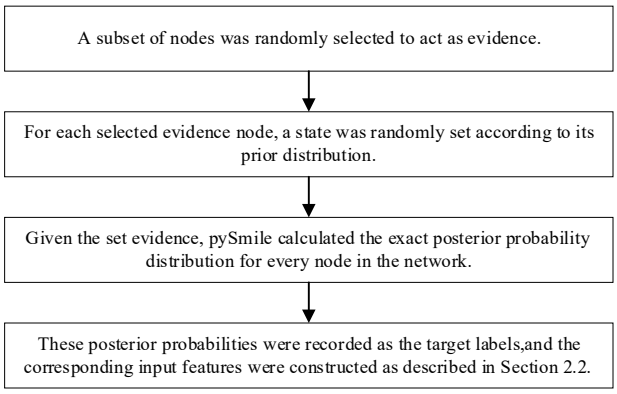


Figure 1 - The generation process for each sample

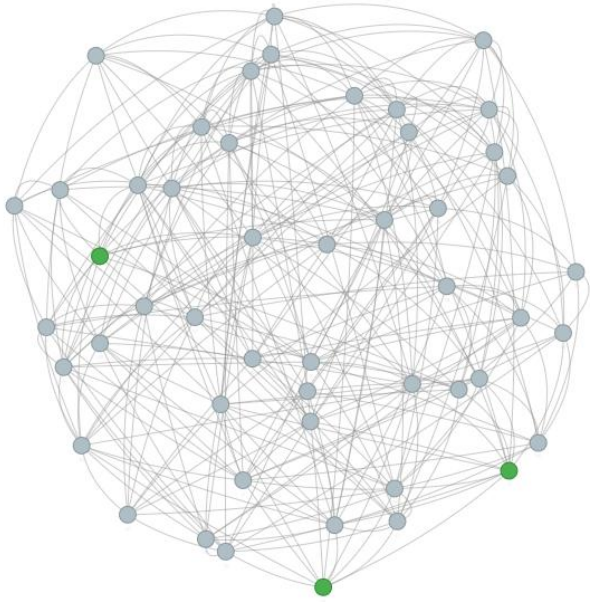


Figure 2 – A sample instance from the generated dataset

2.4. Data Splitting

The full dataset of graph samples was partitioned into training (60%), validation (20%), and testing (20%) sets. Then each of the datasets was loaded in batches of 256.

2.5. Graph Layers

In this paper four different models were developed: MPNN, GCN, GATv2 and Bayesian GNN.

2.5.1. Model 1: MPNN

To establish a performance baseline, a generic Message-Passing Neural Network (MPNN) was implemented. This architecture, formalized by Gilmer et al. [2], provides a unified framework that encompasses many GNN variants. It serves as a powerful, general-purpose model for learning on graph-structured data.

The core of the MPNN operates in a message-passing phase that runs for a fixed number of timesteps, T. In each timestep t, messages are passed between nodes and used to update their hidden states. The process is defined by a message function (M_t) and a vertex update function (U_t).

$$m_v^{t+1} = \sum_{\omega \in N(v)} M_t(h_v^t, h_\omega^t, e_{v\omega}) \quad (1)$$

$$h_v^{t+1} = U_t(h_v^t, m_v^{t+1}) \quad (2)$$

Where h_u^t is the hidden state vector of node u at timestep t, $N(u)$ is the set of neighboring nodes of u, m_u^{t+1} is the aggregated incoming message to node u and M_t *and* U_t are learnable functions, typically implemented as Multi-Layer Perceptrons (MLPs).

While the general MPNN framework is highly expressive, our baseline utilizes a specific, linear variant similar to a Graph Convolutional Network (GCN). This simplifies the general message and update functions into learnable linear transformations. The update rule for a single node u at step t+1 can be expressed as:

$$h_u^{(k)} = \sigma(W_{self}^{(k)} h_u k^{(k-1)} + W_{neigh}^{(k)} \sum_{v \in N(u)} h_v^{(k-1)}) \quad (3)$$

Here, W_{self} and W_{neigh} are learnable weight matrices that transform the node's own representation and the aggregated representation of its neighbors, respectively. σ denotes a non-linear activation function, such as ReLU.

This node-level operation can be expressed more compactly for the entire graph using matrix operations. Let H^t be the matrix containing all node hidden states at step t, and let A be the adjacency matrix of the graph. The update for all nodes becomes:

$$H^{(t+1)} = \sigma((A + I)H^{(t)}W^{(t+1)}) \quad (4)$$

This formulation serves as a flexible but structurally-uninformed baseline. Its performance will be compared against more specialized architectures that may better leverage the principles of probabilistic graphical models.

2.5.2. Model 2: Graph Convolution Neural Network (GCN)

The Graph Convolutional Network (GCN), introduced by Kipf and Welling [3], is a foundational architecture in the GNN landscape. It provides an efficient and scalable method for applying convolutional operations to graph-structured data, drawing inspiration from spectral graph theory. The GCN is not merely a baseline but represents a significant milestone in developing practical deep learning models for graphs.

A MPNN that simply sums neighbor features (as seen in the baseline model with $W_{self} = 0$) can suffer from instability during training. When messages are aggregated using a simple sum, nodes with a high degree will accumulate feature vectors with much larger magnitudes than low-degree nodes. During deep message passing over many layers, this can cause the embeddings to either explode or vanish, making the model highly sensitive to the graph's topology and difficult to optimize.

To solve this, GCN imposes a strong inductive bias by using a fixed, non-learnable message aggregation scheme derived from a first-order approximation of spectral graph convolutions. This clever design avoids the need for expensive spectral decompositions while retaining the benefits of operating on the graph's structural properties.

Instead of a simple sum, GCN employs a symmetrically normalized aggregation. At the node level, the aggregated message m_u^{t+1} for a node u is calculated by averaging the features of its neighbors (and itself), normalized by the degrees of both the source and target nodes:

$$m_u^{t+1} = \sum_{v \in N(u)} \frac{1}{\sqrt{\hat{\deg}(u)\hat{\deg}(v)}} h_v^t \quad (5)$$

In this formulation, $\hat{\deg}(u)$ represents the degree of node u after adding a self-loop. This normalization ensures that the scale of the aggregated message is not skewed by high-degree nodes. The contribution from a neighbor v is down-weighted by both its own degree and the degree of the target node u, leading to a more stable and robust representation.

By incorporating a learnable weight matrix W^t and a non-linear activation function σ(like ReLU), the full GCN layer propagation rule is defined in matrix form as:

$$H^{(t+1)} = \sigma(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}H^{(t)}W^{(t)}) \quad (6)$$

Where $H^{(t)} \in R^{N \times F}$ is the matrix of node features for N nodes at layer t, $\hat{A}$ is the adjacency matrix with self-loops added, $\hat{D}$ is the diagonal degree matrix and W^t is the learnable weight matrix of layer t.

This elegant formulation effectively performs feature transformation and structurally-aware aggregation in a single, efficient matrix multiplication step. The GCN serves as a powerful and widely-used model, and in this study, it represents a more sophisticated baseline with strong theoretical grounding.

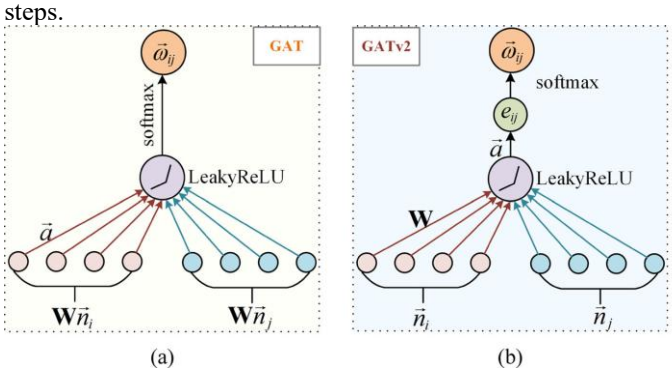
2.6. Model 3: Improved Graph Attention Network (GATv2)

The Improved Graph Attention Network (GATv2), proposed by Brody et al. in 2021[5], is a direct successor to the original GAT [8] architecture. It was designed to address a key theoretical limitation of its predecessor: static attention.

While GAT was a major advance, its attention mechanism computes scores in a static manner, meaning it learns a fixed set of weights for features across all nodes. It cannot change which features to focus on based on the context of the specific node being updated.

GATv2 solves this by computing dynamic attention, which allows the model to be more expressive and flexible. It achieves this by making a simple but powerful change to the order of operations within the attention mechanism, allowing the model to learn relationships that are dependent on both the source and target node in a more complex way. Figure 3 illustrates the difference between GAT and GATv2.

The GATv2 layer can also be understood in a few key steps.



(a) (b)

Figure 3 - Illustration of the graph attention mechanism. A) The attention mechanism of GAT. B) The attention mechanism of GATv2 [6].

The following step-by-step explanation of the GATv2 architecture is adapted from the pedagogical approach presented in 'Hands-On Graph Neural Networks Using Python' by Maxime Labonne [7]

2.6.1. Linear Transformation

This step marks the crucial difference between GAT and GATv2. In the original GAT, the learnable transformation W is applied to the node features h_i and h_j before they are concatenated.

In GATv2, the node features h_i and h_j are first concatenated. Then, a single shared weight matrix W is applied to the combined vector.($[h_i||h_j] \rightarrow W[h_i||h_j]$)

2.6.2. Activation Function

Similar to the original GAT, a non-linear activation function is applied to the output of the linear transformation. The GATv2 paper also uses the Leaky ReLU function to introduce non-linearity and prevent the "dying ReLU" problem. The operation is now performed after the joint linear transformation:

$$e'_{ij} = LeakyReLU(W[h_i||h_j]) \quad (7)$$

2.6.3. Softmax Normalization

The output from the activation function is a vector. To produce a single scalar attention score e_{ij} that represents the importance of node j's message to node i, this vector is multiplied by a learnable attention vector a.

$$e_{ij} = a^T e'_{ij} \quad (8)$$

These raw attention scores are not comparable across different neighborhoods. Therefore, just as in GAT, the softmax function is used to normalize the scores across all neighbors j of node i (including i itself):

$$a_{ij} = softmax_j(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in N_i} \exp(e_{ik})} \quad (9)$$

The resulting a_{ij} values are the final, normalized attention coefficients that can be directly used for message aggregation.

2.6.4. Multi-Head Attention

The learning process of a single self-attention mechanism can be unstable. GATv2 employs the same solution as GAT and the original Transformer: multi-head attention. The process of calculating attention scores (steps 1-3) is repeated multiple times in parallel, each with its own set of learnable parameters ($W^{(k)}$ and $a^{(k)}$).

Each attention "head" k produces its own set of attention scores $a_{ij}^{(k)}$ and, consequently, its own output embedding $h_i'^{(k)}$. These outputs are then combined in one of two ways: Concatenation and averaging.

Concatenation is used for hidden layers where the embeddings from all heads are concatenated:

$$h_i' = ||_{k=1}^{K} \left(\sum_{j \in N_i} a_{ij}^{(k)} W^k H_j \right) \quad (10)$$

Averaging is used for the final output layer and the embeddings are averaged:

$$h_j' = \frac{1}{K} \sum_{k=1}^{K} \left(\sum_{j \in N_i} a_{ij}^{(k)} W^k H_j \right) \quad (11)$$

In the models developed for this research, we set the number of attention heads to 8 (K=8). For all hidden layers, the feature vectors produced by these heads are concatenated, creating a larger feature vector for the next layer. In the final output layer of the network, the 8 head outputs are averaged to compute the final probability distribution for each node.

2.7. Model 4: Bayesian Graph Neural Network (BGNN)

While the GCN and GAT architectures can learn to approximate the posterior probabilities within a Bayesian Network, they are fundamentally deterministic models. They learn a single set of weights and thus produce a single point estimate of the posterior for a given evidence configuration. This approach, while potentially accurate, fails to capture the GNN's own epistemic uncertainty—the uncertainty in its parameters from being trained on finite data.

To address this limitation and create a framework more aligned with the probabilistic nature of the original problem, the final model is a Bayesian Graph Neural Network (BGNN).

Bayesian statistics offer a formal way to understand and quantify this uncertainty. By learning a full probability distribution over its weights, the BGNN can provide a more principled and complete approximation of Bayesian inference. This allows it to not only predict posterior probabilities but also to quantify its own confidence in those predictions, a critical aspect of true Bayesian reasoning [9].

The standard approach is to approximate a minimal cost point estimate of the network parameters θˆ, i.e., a single value for each parameter (Figure 4a).

Stochastic neural networks are a type of ANN built by introducing stochastic components into the network. This is performed by giving the network either a stochastic activation (Figure 4b) or stochastic weights (Figure 4.c) to simulate multiple possible models θ with their associated probability distribution $p(\theta)$

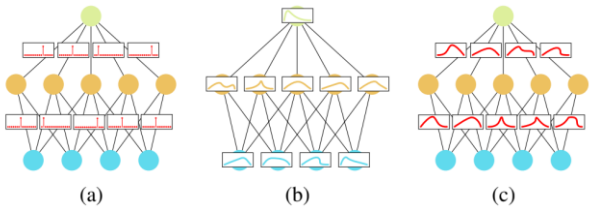


Figure 4 - (a) Point estimate neural network, (b) stochastic neural network with a probability distribution for the activations, and (c) stochastic neural network with a probability distribution over the weights.[9]

Training a BGNN involves computing the posterior distribution of the weights given the data, $p(\theta \mid D)$. As this is intractable for deep neural networks, Variational Inference (VI), have been used which is a scalable approximation method.

The core idea of VI is to define a simpler, parameterized distribution, the variational posterior $q_\phi(\theta)$, and optimize its parameters ϕ to approximate the true posterior.

This is achieved by minimizing the Kullback-Leibler (KL) divergence between the two distributions.

For two continuous probability distributions, $P(x)$ and $Q(x)$, the KL divergence of Q from P is defined as:

$$D_{KL}(P||Q) = \int_{-\infty}^{\infty} p(x) \log\left(\frac{p(x)}{q(x)}\right) dx \quad (12)$$

In the context of Bayesian variational inference, the formula measures the difference between the approximate variational posterior $q_\phi(H)$ and the true posterior P(H|D). the formula is:

$$D_{KL}(q_\phi||P) = \int_H q_\phi(H') \log\left(\frac{q_\phi(H')}{P(H'|D)}\right) dH' \quad (13)$$

Directly minimizing the KL-divergence is not feasible, so in practice, we maximize a more convenient objective function called the Evidence Lower Bound (ELBO). Maximizing the ELBO is equivalent to minimizing the KL-divergence and is defined as:

$$\mathcal{L}_{ELBO}(\phi) = \underbrace{E_{q_\phi(\theta)}[\log p(D|\theta)]}_{\text{The Expected Log-Likelihood}} - \underbrace{D_{KL}(q_\phi(\theta)||p(\theta))}_{\text{The KL Divergence}} \quad (14)$$

This objective consists of two terms:

1. The Expected Log-Likelihood: The data-fit term, which encourages the model to explain the observed data.
2. The KL Divergence: A regularization term that penalizes the model if the learned variational posterior $q_\phi(\theta)$ strays too far from a predefined prior distribution $p(\theta)$ over the weights.

The BGNN architecture is implemented using the Bayes-by-Backprop algorithm, where standard layers are replaced with Bayesian layers.

The variational posterior for each weight and bias, $q_\phi(\theta)$, is modeled as a fully factorized Gaussian distribution.

A standard Gaussian distribution, $p(\theta) = N(0,1)$, is used as the prior, a common default choice.

To enable gradient-based optimization, each Bayesian layer uses the reparameterization trick. The stochastic output activation computed as a deterministic function of the input x and a random noise variable $\epsilon \sim N(0,1)$, allowing gradients to flow through the network. The output activation is computed as:

$$\mu_a = x\mu_W^T + \mu_b \quad (15)$$
$$\sigma_a^2 = x^2(\sigma_W^2)^T + \sigma_b^2 \quad (16)$$
$$a = \mu_a + \sigma_a \odot \epsilon \quad (17)$$

Here (μ_W, σ_W^2) and (μ_b, σ_b^2) are the learnable means and variances of the wights and biases.

Making a prediction requires marginalizing over the posterior to get the predictive distribution $p(y \mid \mathbf{x}, D)$. This integral is intractable:

$$p(y|\mathbf{x}, D) = \int p(y|\mathbf{x}, \theta)p(\theta|D)d\theta \quad (18)$$

This integral is approximated using Monte Carlo (MC) sampling. Multiple forward passes are performed, each time drawing a new set of weights $\{\theta_1 \dots \theta_N\}$ from learned posterior $q_\phi(\theta)$. The resulting predictions are aggregated to produce the final output and its uncertainty, a process known as model averaging [9].

2.8. Common Network Architecture

To ensure a robust comparison, each of the four models (MPNN, GCN, GATv2, and BGNN) was constructed based on a similar multi-layer GNN template shown in Figure 5. However, each architecture was then individually tuned to achieve its optimal performance on the task. This empirical tuning process resulted in slight variations in depth and the inclusion of model-specific techniques. A common feature across all models is the use of a hidden dimension size of 64.

The MPNN and GCN models were implemented with a deeper architecture of four hidden layers.

For the GCN model specifically, performance was substantially improved by incorporating a Jumping Knowledge (JK) connection layer at the end of the network. JK connections aggregate the outputs from all preceding hidden layers, allowing the model to better leverage information from different neighborhood depths and mitigate the over-smoothing effects common in deep GCNs.

The GATv2 and BGNN models achieved their best performance with a slightly shallower architecture of three hidden layers.

To improve training stability and gradient flow in the BGNN model, residual connections were included. In this architecture, the residual connection was applied after the ReLU activation function.

These final architectural choices were the result of an iterative tuning process aimed at finding the strongest configuration for each model before the final evaluation. Key components of the architecture include:

2.8.1. Residual Connections

Each hidden layer incorporates a skip connection, where the input to the layer is added to its output. These connections are crucial for training deep networks as they mitigate the vanishing gradient problem and allow the model to learn identity mappings more easily. Residual connections solve vanishing gradient problem by providing an alternative, direct path for the gradient to flow.

Architecturally, instead of a layer's output being just the result of a transformation F(x), it becomes the transformation plus the original input, x

2.8.2. Batch Normalization

A BatchNorm1d layer is applied after convolution and before the activation function. This standardizes the inputs to the next layer, which stabilizes the training process and often improves convergence speed.

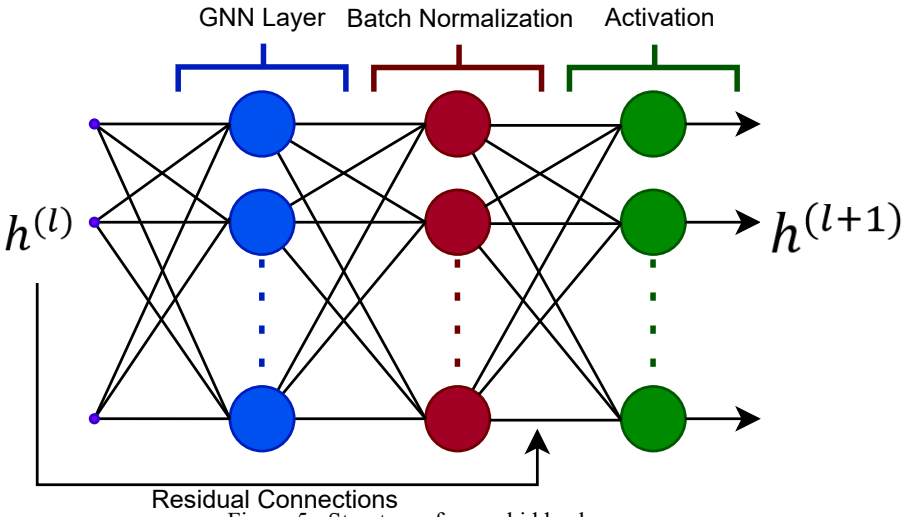


Figure 5 - Structure of every hidden layer

During training, the weights in each layer of a neural network are constantly being updated. This means that the distribution of the outputs from one layer (which are the inputs to the next layer) is continuously changing. This phenomenon is called internal covariate shift.

This shifting distribution makes training difficult because each layer must constantly adapt to a new input distribution. It's like trying to learn a function when the domain and scale of the input data are always moving. This forces developers to use lower learning rates and be very careful with weight initialization, which slows down the training process significantly.

2.8.3. Activation Function

For three of the evaluated models—MPNN, GCN, and BGNN—the standard Rectified Linear Unit (ReLU) function was used as the non-linear activation in the hidden layers.

An exception was made for the GATv2 model to closely follow its original implementation. For this architecture, the Leaky ReLU activation function was employed. Leaky ReLU is a variant that allows a small, non-zero gradient when the unit is not active. The negative slope (α) for this function was set to 0.01.

The computational flow for each hidden layer follows this sequence:

1. Input features from the previous layer $h^{(l)}$
2. Graph convolution operation (MPNN, GCN, GATv2, or Bayesian), producing h'_{conv}
3. Batch Normalization $h'_{bn} = BatchNorm(h'_{conv})$
4. Residual Connection $h'_{res} = h'_{bn} + h^{(l)}$
5. Activation $h^{(l+1)} = (Leaky)ReLU(h'_{res})$

Finally, the output layer takes the features from the last hidden layer and projects them to the final output dimension. The network applies a LogSoftmax function to this final output to produce a logarithmic probability distribution.

2.9. Loss function

The primary objective of all evaluated models is to predict the posterior probability distribution for each node in the Bayesian Network, given a set of observations. Since the ground-truth posterior for every node is known from the pre-computed dataset, this task can be framed as learning a mapping from input features to a target probability distribution.

To measure the fidelity of the model's predicted distribution against the true posterior, the Kullback-Leibler (KL) Divergence was selected as the loss function. KL Divergence is a natural choice from information theory that quantifies how one probability distribution (Q) diverges from a second, reference probability distribution (P). For discrete distributions over N states, it is defined as:

$$D_{KL}(P||Q) = \sum_{i=1}^{N} P(i) \log\left(\frac{P(i)}{Q(i)}\right) \quad (19)$$

In which P is the true posterior probability distribution for a node and Q is the distribution predicted by GNN model.

Minimizing this loss function drives the model's predicted distribution (Q) to become as close as possible to the true posterior distribution (P). It is important to note that the loss is only computed for nodes that are unobserved in a given sample; the model is not penalized on its predictions for nodes whose states are provided as evidence.

2.10. Evaluation Metrics

To provide a comprehensive assessment of each model's performance, a suite of evaluation metrics was employed. While the KL Divergence serves as the primary loss function for training, relying on it alone for evaluation can be insufficient. Therefore, additional metrics were chosen to capture different aspects of predictive accuracy, focusing on the similarity between the predicted probability distributions and the ground-truth posterior distributions from the dataset.

For the deterministic models (MPNN, GCN, GATv2), the predicted distribution is the direct output of a single forward pass. For the Bayesian GNN (BGNN), the final predicted distribution is the average result of multiple Monte Carlo samples, which approximates the Bayesian model average. All metrics are calculated only on the unobserved nodes in the test set.

2.10.1. Cosine Similarity

Cosine Similarity measures the cosine of the angle between the predicted probability vector (q) and the true probability vector (p). This metric is particularly useful because it is insensitive to the magnitude of the vectors, focusing instead on their orientation. In this context, it evaluates whether the model correctly captures the relative proportions of probabilities across the different states, even if the absolute predicted probabilities are not perfectly calibrated.

$$Cosine\ Similarity = \frac{p \cdot q}{||p||\,||q||} \quad 20$$

The score ranges from -1 to 1, where 1 indicates that the predicted and true probability vectors are perfectly aligned.

2.10.2. Pearson Correlation Coefficient

The Pearson Correlation Coefficient (ρ) measures the linear relationship between the elements of the predicted probability vector (q) and the true probability vector (p). It assesses whether an increase in the true probability of a given state corresponds to a consistent, linear increase in the predicted probability for that state. This provides insight into the structural correctness of the model's predictions.

It is calculated as the covariance of the two vectors divided by the product of their standard deviations:

$$\rho(\mathbf{p}, \mathbf{q}) = \frac{cov(\mathbf{p},\mathbf{q})}{\sigma_p \sigma_q} \quad (21)$$

The score ranges from -1 to 1. A value of 1 signifies a perfect positive linear correlation, indicating that the model's predictions align linearly with the true posteriors.

2.11. Training the model

All four models were trained on the same dataset consisting of 5,000 samples, which was split into training, validation, and test sets. The training was conducted for 150 epochs. During this process, the model's performance was monitored by recording the following metrics every 10 epochs:

- Mean KL Divergence loss on the training set.
- Mean KL Divergence loss, cosine similarity, and Pearson correlation coefficient on the validation set.

To ensure the best possible version of each model was used for final testing, the model checkpoint that achieved the lowest KL Divergence loss on the validation set was saved.

The evaluation involved both a comprehensive quantitative analysis and a targeted qualitative case study.

The best-performing checkpoint for each model was applied to the unseen test set. The overall performance was measured by calculating the mean KL Divergence loss, cosine similarity, and Pearson correlation coefficient across all unobserved nodes in the test dataset. These metrics provide an aggregate measure of each model's predictive accuracy and ability to approximate the true posterior distributions.

To allow for a more direct and interpretable comparison, a single representative data sample was selected from the test set. For a specific node within this sample, the predicted posterior distribution from each of the four models was calculated and visualized, allowing for a side-by-side comparison against the ground-truth posterior.

3. Results and discussions

This section analyzes the performance of the four GNN models by presenting their training dynamics, quantitative test results, and a qualitative case study.

The GCN model demonstrated the most efficient and effective learning process. Its loss curves, shown in Figure 6B, are characterized by a rapid and sharp decrease, with the model converging to a very low validation loss within the first 30 epochs. This superior performance was consistent across all validation metrics, where it quickly achieved and maintained the highest scores for Cosine Similarity and Pearson Correlation (Figure A and B).

A notable feature of the GCN's training is that the validation loss is consistently lower than the training loss. This is a positive indicator that the regularization used during training is effective. The training loss is calculated with an active regularization penalty (like L2 weight decay), making it artificially higher.

The proposed BGNN also showed strong performance and excellent generalization. While its convergence was more gradual than the GCN's, it achieved a highly competitive final validation loss (Figure C). A key feature of its training is the proximity of the validation and training loss curves (Figure 6D), which indicates a strong resistance to overfitting—a direct benefit of its Bayesian formulation that regularizes the model by learning a distribution over its weights.

The GATv2 model, while an improvement over the baseline, did not match the performance of the GCN or

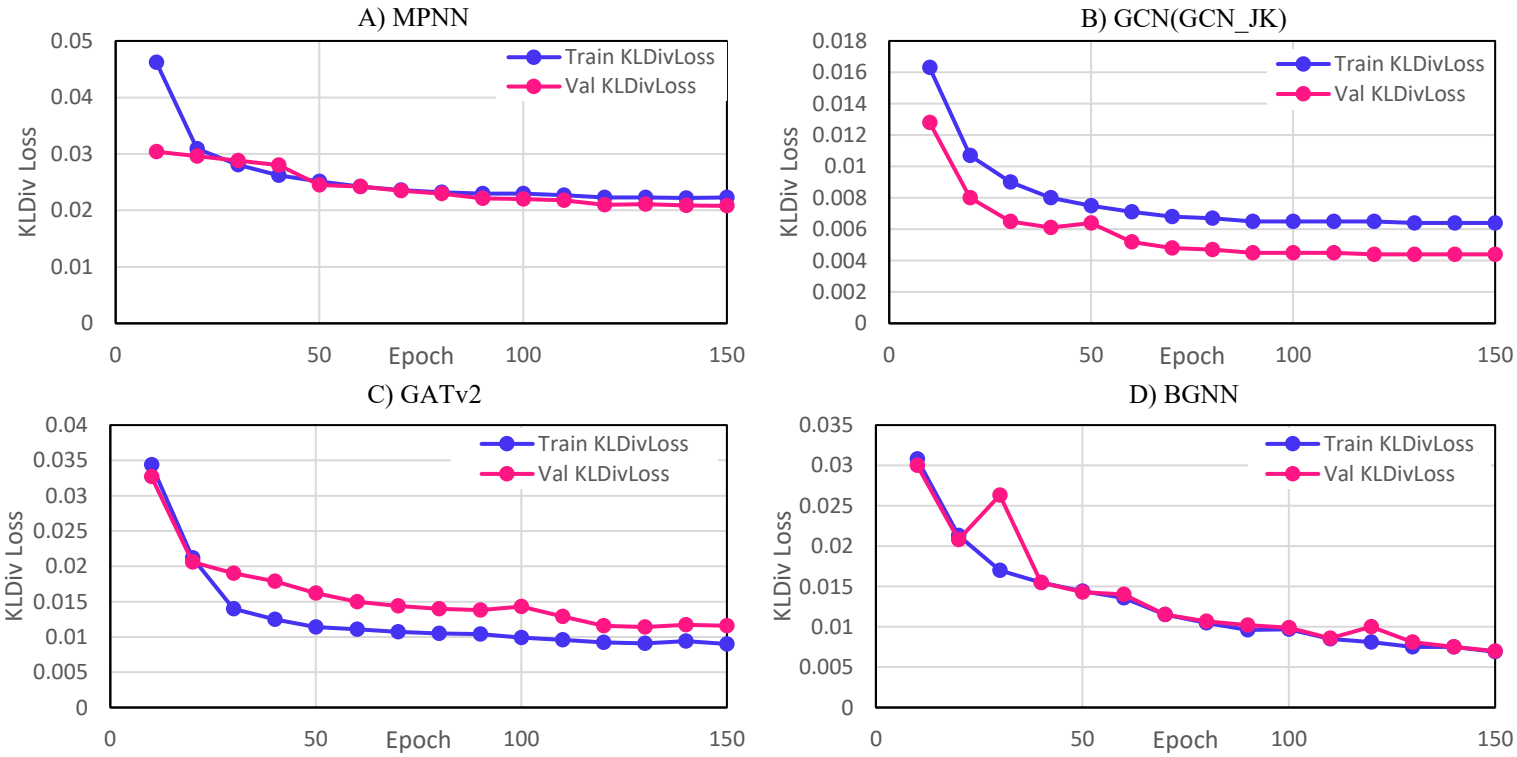


Figure 6 - Training and validation Kullback-Leibler (KL) Divergence loss curves for the four GNN models: (A) MPNN, (B) GCN, (C) GATv2, and (D) BGNN

BGNN. Its training curve (Figure 6C) displays a noticeable gap between the training and validation losses, suggesting a tendency to overfit, which is a common challenge for more complex architectures.

The MPNN served as a solid baseline, showing stable convergence (Figure 6A), but was clearly outperformed by the more advanced models on all validation metrics.

The final performance on the unseen test set, summarized in Table 1 aligns with the trends observed during validation. The GCN solidified its position as the top-performing model, achieving the best scores across all three metrics.

The BGNN confirmed its status as a strong runner-up with highly competitive results. Notably, the GATv2's test performance fell below the baseline MPNN on the Pearson Correlation metric, reinforcing the hypothesis that it overfitted the training data.

Table 1 – Final evaluation result of test dataset for different models

Model	KL diversion	Cosine Similarity	Pearson Correlation
MPNN	0.0210	0.9859	0.6660
GCN	0.0045	0.9967	0.7560
GATv2	0.0114	0.9913	0.6454
BGNN	0.0068	0.9948	0.6898

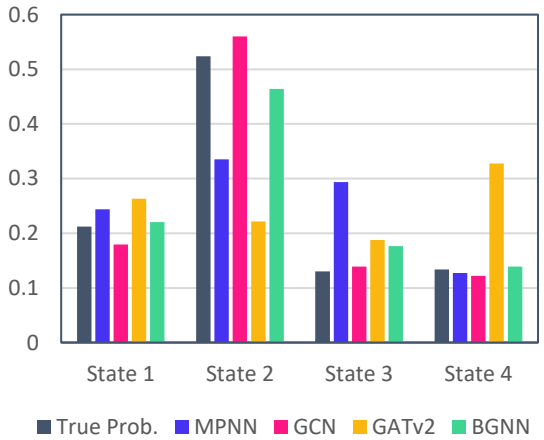


Figure 7 – Comparison of predicted posterior probabilities for node 10

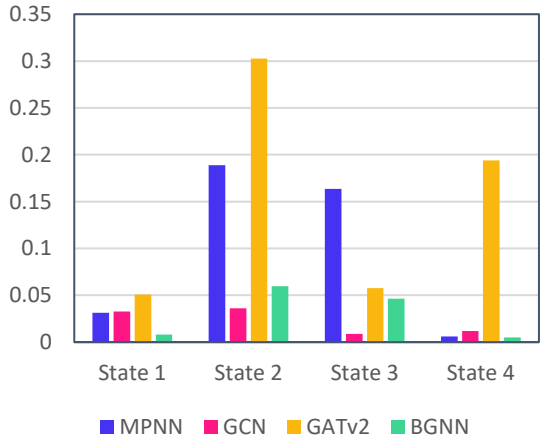


Figure 8 – Comparison of model prediction errors for node 10.

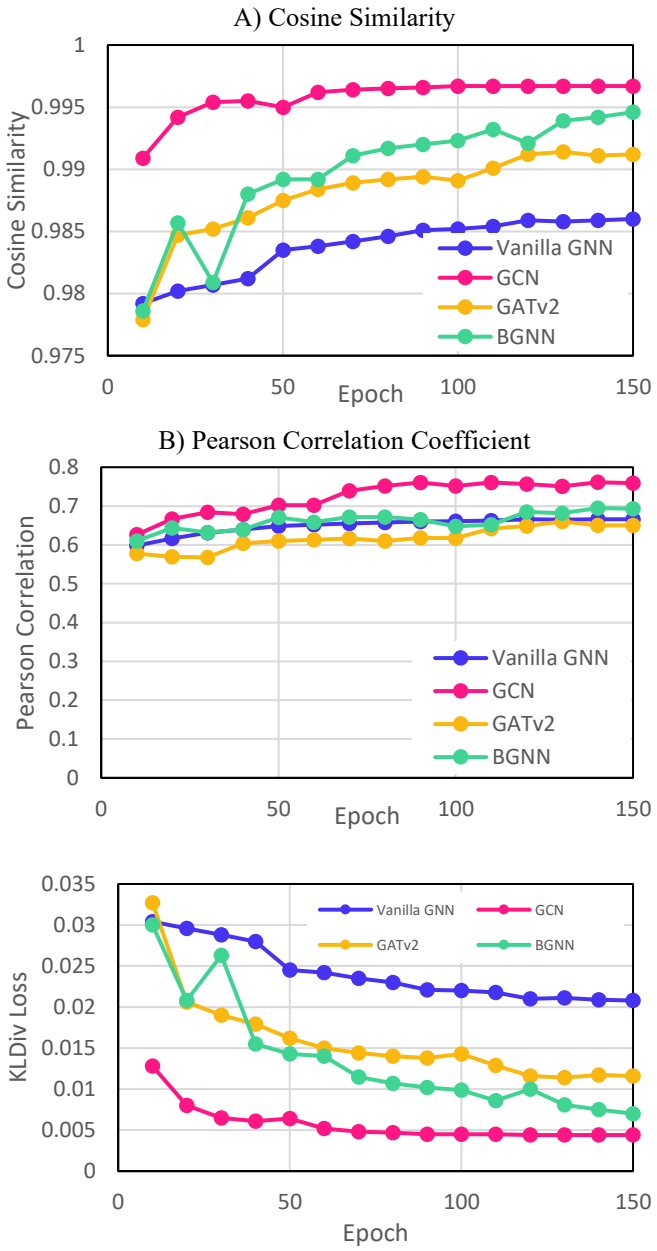


Figure 9 - Comparison of model performance on the validation set over 150 training epochs, evaluated using three different metrics: (A) Cosine Similarity, (B) Pearson Correlation Coefficient, and (C) KL

A qualitative case study on a single node provides a visual representation of these quantitative results. In Figure 7, the bar chart shows the GCN's predicted distribution almost perfectly overlapping with the true posterior, with the BGNN also providing a very close approximation. The corresponding prediction errors, quantified in Figure 8, are minimal for these two models. This visual evidence reinforces their superior accuracy, although it is important to reiterate that a general conclusion cannot be drawn from a single sample.

4. Conclusion

This research successfully demonstrated that Graph Neural Networks provide a powerful and viable framework for approximating probabilistic inference in Bayesian Networks. GNNs can learn a generalized inference function directly from data, offering a scalable alternative to traditional, often computationally expensive, inference algorithms. However, the performance is an approximation and is highly dependent on the chosen GNN architecture, the network topology, and the training data.

In this study, a novel Bayesian Graph Neural Network (BGNN) was developed and evaluated against several deterministic benchmarks. While the GCN model achieved the highest point-estimate accuracy, the proposed BGNN delivered highly competitive performance that significantly surpassed the baseline MPNN and the more complex GATv2. The primary contribution of the BGNN is its principled approach to uncertainty quantification. By learning a distribution over its weights, it provides not just a prediction but also a measure of confidence, which is a critical feature for a fundamentally probabilistic task.

A promising avenue for future work is to combine the strengths of the best-performing architectures. Developing Bayesian versions of GCN or GATv2 could create even more powerful models, merging the effective structural biases and normalization of the GCN with the robust regularization and uncertainty modeling of a Bayesian approach. Ultimately, this work establishes a strong foundation for using GNNs in probabilistic modeling and highlights the significant potential of Bayesian GNNs in creating more reliable and uncertainty-aware intelligent systems.

5. Nomenclature

A	Adjacency matrix of the graph
$\tilde{A}$	Adjacency matrix with self-loops added
a_{ij}	Normalized attention coefficient for node j to i.
$D_{\max}$	The maximum number of states across all nodes
$\widetilde{D}$	The diagonal degree matrix corresponding to $\tilde{A}$
e_{ij}	Unnormalized attention score for node j to i.
$H^{(t)}$	Matrix containing the hidden states of all nodes at layer/timestep t
$h_v^{(t)}$	The hidden state vector of a node v at layer/timestep t.
I	Identity matrix.
K	The number of parallel attention heads in Multi-Head Attention.
m_v^{t+1}	The aggregated message vector for node v at layer/timestep $t+1$
$N(v)$	The set of neighboring nodes of node v
$p(\cdot)$	A true probability distribution
$q(\cdot)$	An approximate probability distribution
$p(\theta)$	The prior probability distribution weights
$p(\theta\|D)$	The true posterior probability distribution of weights given data D
$q_\phi(\theta)$	The approximate variational posterior
T	The total number of message-passing timesteps
$W^{(t)}$	The input feature vector for node v
X_v	A learnable weight matrix at layer t
Y_v	The target output (label) vector for node v

Greek Symbols

ϵ	A random noise variable sampled from a standard normal distribution.
θ	A general representation for the parameters
μ_W, μ_b	The learnable means of the weight and bias distributions in a Bayesian layer
$\rho(p,q)$	The Pearson Correlation Coefficient
$\sigma(\cdot)$	A non-linear activation function (e.g., ReLU)
σ_W^2, σ_b^2	The learnable variances of the weight and bias distributions
ϕ	The learnable parameters of the variational posterior distribution $q_\phi(\theta)$

6. References

[1] Koller, D., and Friedman, N., 2009. *Probabilistic graphical models: principles and techniques*. MIT press, Cambridge, MA.

[2] In proceedings, Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., and Dahl, G. E., 2017. "Neural Message Passing for Quantum Chemistry". In *Proceedings of the 34th International Conference on Machine Learning*, PMLR, pp. 1279–1288. Paper number arXiv:1704.01212

[4] In proceedings, Kipf, T. N., and Welling, M., 2017. "SEMI-SUPERVISED CLASSIFICATION WITH GRAPH CONVOLUTIONAL NETWORKS". In *International Conference on Learning Representations (ICLR)*. Paper number arXiv:1609.02907.

[5] In proceedings, Brody, S., Alon, U., and Yahav, E., 2022. "HOW ATTENTIVE ARE GRAPH ATTENTION NETWORKS?". In *International Conference on Learning Representations (ICLR)*. Paper number arXiv:2105.14491.

[6] Liu, J., Xie, F., Zhang, Q., Lyu, Q., Wang, X., and Wu, S., 2024. "A multisensory time-frequency features fusion method for rotating machinery fault diagnosis under nonstationary case". *Journal of Intelligent Manufacturing*, 35(7), October, pp. 3197–3217.

[7] Labonne, M., 2023. *Hands-On Graph Neural Networks Using Python: Practical techniques and architectures*. Packt Publishing, Birmingham, UK.

[8] In proceedings, Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y., 2018. "Graph Attention Networks". In *International Conference on Learning Representations (ICLR)*. Paper number arXiv:1710.10903.

[9] Jospin, L. V., Buntine, W., Boussaid, F., Laga, H., and Bennamoun, M., 2022. "Hands-on Bayesian Neural Networks-A Tutorial for Deep Learning Users". *IEEE Computational Intelligence Magazine*, 17(2), May, pp. 20–38.

[10] Shadabfar, M., Mahsuli, M., Zhang, Y., Xue, Y., Ayyub, B. M., Huang, H., and Medina, R. A., 2021. "Resilience-Based Design of Infrastructure: Review of Models, Methodologies, and Computational Tools". ASCE-ASME Journal of Risk and Uncertainty in Engineering Systems, Part A: Civil Engineering, 8(1), November, p. 03121004.